PBFL: A Privacy-Preserving Blockchain-Based Federated Learning Framework With Homomorphic Encryption and Single Masking

Baofu Han, Bing Li, Raja Jurdak, *Senior Member, IEEE*, Peiyun Zhang, *Senior Member, IEEE*, Hao Zhang, Pan Feng, and Chau Yuen, *Fellow, IEEE*

***Abstract*—Federated learning (FL) has emerged as a promising paradigm for secure data sharing in Industrial Internet of Things (IIoT), enabling collaborative model training without direct exchange of raw data. However, recent studies have shown that FL still suffers from privacy vulnerabilities, where adversaries can reconstruct sensitive information by analyzing shared model parameters. Although several privacy-preserving FL (PPFL) schemes have been proposed to address these challenges, they primarily focus on protecting local model privacy, with limited attention to protecting global model confidentiality during aggregation. Additionally, their reliance on centralized aggregation servers introduces risks of single points of failure. To address these challenges, we propose a novel privacy-preserving blockchain-based FL framework (PBFL) that integrates blockchain, homomorphic encryption (HE), and a single masking. Specifically, PBFL employs HE to enable secure model training within the ciphertext domain, ensuring global model confidentiality. The single masking technique allows clients to apply unique random masks to their encrypted local model updates, enabling secure aggregation while preserving local privacy. Additionally, PBFL leverages blockchain for decentralized aggregation and encrypted model storage, effectively mitigating the risks associated with centralized servers. Experimental results demonstrate that PBFL achieves comparable model accuracy to state-of-the-art solutions while providing enhanced privacy protection. Furthermore, even with a client dropout rate of up to 30%, PBFL outperforms other blockchain-based PPFL methods in terms of computational and communication efficiency.**

***Index Terms*—Blockchain, federated learning (FL), homomorphic encryption (HE), Industrial Internet of Things (IIoT), privacy-preserving.**

Received 23 September 2024; revised 26 November 2024; accepted 28 December 2024. Date of publication 1 January 2025; date of current version 9 May 2025. This work was supported in part by the Postgraduate Research and Practice Innovation Program of Jiangsu Province under Grant KYCX23_0320; in part by the China Scholarship Council under Grant 202406090038; and in part by the Cyber Security School Student Innovation Funding Program. *(Corresponding author: Bing Li.)*

Baofu Han and Pan Feng are with the School of Cyber Science and Engineering, Southeast University, Nanjing 211102, China (e-mail: hanbaofu@seu.edu.cn; fp_1022@seu.edu.cn).

Bing Li and Hao Zhang are with the School of Microelectronics, Southeast University, Nanjing 211102, China (e-mail: bernie_seu@seu.edu.cn; 220226283@seu.edu.cn).

Raja Jurdak is with the School of Computer Science, Queensland University of Technology, Brisbane, QLD 4001, Australia (e-mail: r.jurdak@qut.edu.au).

Peiyun Zhang is with the Engineering Research Center of Digital Forensics of Ministry of Education and the School of Computer Science, Nanjing University of Information Science and Technology, Nanjing 210044, China (e-mail: zpy@nuist.edu.cn).

Chau Yuen is with the School of Electrical and Electronic Engineering, Nanyang Technological University, Singapore (e-mail: chau.yuen@ntu.edu.sg).

Digital Object Identifier 10.1109/JIOT.2024.3524632

I. Introduction

THE CONVERGENCE of 5G networks and edge computing technologies has accelerated the evolution of Industrial Internet of Things (IIoT), enabling vast numbers of industrial devices to interconnect in real time and generate massive volumes of data [1], [2], [3]. By leveraging the high bandwidth and low latency of 5G, combined with the distributed processing power of edge computing, IIoT has been widely adopted in various scenarios, such as industrial automation, smart manufacturing, and predictive maintenance [4]. Machine learning (ML), with its powerful data analysis capabilities, can effectively utilize these industrial data to achieve automated decision-making and enhance the level of industrial intelligence [5]. However, traditional ML methods typically require collecting large amounts of raw data from IIoT devices and transferring it to a data center for centralized model training. Due to the sensitive nature of industrial data, this centralized approach introduces privacy risks. Therefore, achieving high-precision model training while protecting data privacy has become a critical challenge [6].

Federated learning (FL), as an emerging distributed ML paradigm, is increasingly recognized as a promising solution to resolve the inherent tradeoff between protecting data privacy and improving model performance [7]. In FL, multiple devices collaborate to train an ML model using their local data and transmit only model updates to a central server. The central server then aggregates these updates to generate a global model, thereby preserving data privacy by avoiding the transfer of raw data [8]. Despite this, recent studies have shown that FL systems remain vulnerable to privacy leakage, as adversaries can conduct inference attack to reconstruct sensitive information from the shared model parameters [9], [10].

To address these challenges, some privacy-preserving FL (PPFL) schemes have been proposed in recent years, which can be primarily categorized into three types: 1) differential privacy (DP); 2) homomorphic encryption (HE); and 3) secure aggregation (SA) [11]. For example, Chen et al. [12] introduced a lightweight privacy-preserving mechanism based on DP, which achieves ϵ-privacy guarantees by injecting

2327-4662 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:04:08 UTC from IEEE Xplore. Restrictions apply.

carefully calibrated noise into model updates. However, it faces a tradeoff between privacy preservation and model accuracy. To overcome this limitation, Fan et al. [13] and Li et al. [14] developed HE-based PPFL schemes that enable secure computation directly on encrypted model updates, thereby preserving model privacy. Nevertheless, these methods require all clients to share the same private key, making the entire system vulnerable if any clients' key is breached. As a promising alternative, SA-based methods ensure model privacy through masking techniques, where the random values injected by clients are effectively canceled out during aggregation, maintaining the accuracy of the global model. For instance, Fu et al. [15] proposed an SA-based PPFL scheme that employs double-masking techniques to protect local model privacy. Furthermore, recent SA-based PPFL frameworks [16], [17], [18] incorporate secret sharing techniques to address client dropout issues due to connectivity problems or power outages.

Despite these advancements, SA-based PPFL schemes still face several key challenges. First, existing schemes primarily focus on protecting local model privacy while lacking effective privacy protection mechanisms for the global model, which may not meet the stringent privacy requirements in privacy-sensitive FL applications. Second, although existing protocols support client dropout scenarios, they require additional communication rounds among online clients for mask reconstruction of dropped clients, posing a significant burden on devices with resource-limited clients. Third, most SA-based PPFL schemes rely on a centralized server for global aggregation, which poses a potential risk of single point of failure [19]. To the best of our knowledge, although some studies [20], [21], [22], [23] have proposed solutions to address some of these challenges, there is currently no work that simultaneously addresses all three challenges in SA-based PPFL.

To address the aforementioned challenges, we propose the PBFL, a novel privacy-preserving blockchain-based FL framework that integrates blockchain, HE, and a single masking. First, PBFL replaces the centralized server with a blockchain as a public platform, enabling decentralized model storage and global aggregation. Second, PBFL utilizes Paillier encryption algorithm to build a global model privacy protection mechanism, allowing clients to directly perform local model training in the encrypted domain, thereby ensuring the privacy of the global model during the training process. Furthermore, PBFL combines a single-mask technique based on homomorphic pseudorandom generator (HPRG), which not only protects local model updates but also eliminates the need for mask reconstruction due to client exits during global aggregation, thereby significantly reducing system communication overhead. The main contributions of our work are summarized as follows.

1) We propose the PBFL framework that leverages blockchain technology to guarantee the reliability and traceability of the training process. Furthermore, to achieve a balance between system security and FL efficiency, we choose the Algorand consensus mechanism to conduct the global aggregation.
2) We develop a privacy-preserving scheme that integrates HPRG-based single masking with homomorphic Shamir secret sharing and Paillier encryption. This scheme ensures the privacy of local and global models while supporting dynamic client withdrawal without complex mask reconstruction, significantly reducing system communication overhead.
3) Through comprehensive security analysis, we prove the PBFL's effectiveness in ensuring privacy. Furthermore, we conduct extensive experiments, and the results demonstrate that even with a client dropout rate of up to 30%, PBFL outperforms other blockchain-based PPFL (BPFL) methods in terms of computational and communication efficiency.

The remainder of this article is organized as follows. Section II reviews related work in areas relevant to our study. Section III introduces the preliminary knowledge. Section IV outlines the system model, design goals, and security assumption. Section V presents a detailed description of our scheme. Section VI gives the security analysis. Section VII presents performance evaluation. Finally, Section VIII concludes this article.

II. Related Work

Our research involves two areas: 1) PPFL and 2) blockchain-based FL. This section will briefly review the latest research progress in these two fields.

A. Privacy-Preserving FL

Although FL provides a certain degree of data privacy, it remains susceptible to inference attacks, where malicious adversaries can extract private information from local model updates. To protect the privacy of local models, numerous PPFL schemes have been proposed in recent years, which can be categorized into three types: DP, HE, and SA.

DP: The lightweight nature of DP has led to its widespread adoption in FL to protect user privacy. For example, Wang et al. [24] and Xiong et al. [25] proposed DP-based PPFL schemes by adding random noise to local model updates, effectively defending against privacy inference attacks. However, these methods uniformly allocate the privacy budget on the server side, making it difficult to achieve personalized protection. To address this, Wei et al. [26] proposed a user-level DP algorithm that enables mobile devices to autonomously adjust their privacy levels. Despite this advance, Khoa et al. [27] observed that DP-based PPFL methods still face a tradeoff between privacy budget and model accuracy.

HE: The unique advantage of HE is that it can perform calculations on encrypted data without decryption, thereby providing stronger privacy protection without compromising model accuracy. Shen et al. [28] developed an HE-based PPFL framework that encrypts model parameters before transmission, preventing adversaries from accessing raw model updates. Further optimizing HE's efficiency, Zhang et al. [29] introduced BatchCrypt, which significantly reduced both encryption and communication costs in cross-device FL while maintaining privacy guarantees. Nevertheless, these methods rely on a shared private key for all clients, exposing the entire system to risk if a key is compromised. To mitigate

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:04:08 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARATIVE ANALYSIS OF EXISTING PPFL METHODS AND THE PROPOSED PBFL METHOD

Schemes	Technology	Local model privacy	Global model privacy	Dropout	Efficiency	Blockchain
Qi et al. [41]	DP	✓	✗	✓	High	✓
ShieldFL. [30]	HE	✓	✗	✗	Low	✗
Fang et al. [43]	Double Masking	✓	✗	✓	Low	✓
PrivFL. [32]	HE + SA	✓	✓	✓	High	✗
Yang et al. [34]	Single Masking + HE	✓	✗	✓	Low	✗
Ma et al. [42]	Double Masking	✓	✗	✗	Low	✓
PBFL	Single Masking + HE	✓	✓	✓	High	✓

this, Ma et al. [30] introduced ShieldFL, a defense strategy employing double-trapdoor HE. ShieldFL ensures that even if one server is compromised, attackers cannot obtain complete privacy information. However, the effectiveness of ShieldFL relies on the assumption of noncollusion between two servers, which may be challenging in practical applications.

SA: In contrast to DP and HE approaches, SA-based PPFL schemes achieve privacy preservation through random masking techniques. Bonawitz et al. [31] proposed the first SA-based PPFL scheme, which ingeniously combined double masking and secret sharing techniques to protect privacy. Mandal and Gong [32] proposed PrivFL, a PPFL approach that employs HE and SA mechanisms to protect data and model privacy in regression training. Building upon [31], Wang et al. [16] and Xu et al. [17] proposed verifiable PPFL schemes that ensure both privacy protection and model integrity verification. However, these methods suffered from high communication overhead during client dropouts, as additional communication rounds are required for mask reconstruction. To overcome this limitation, Zhao et al. [33] developed a single-mask scheme based on the computational Diffie–Hellman assumption, which partially reduces communication costs. However, this method fails to handle user dropouts effectively. Liu et al. [20] introduced an improved single-mask SA protocol that utilizes HPRG and Diffie–Hellman assumption to effectively deal with the client dropouts, but performance bottlenecks remain due to the need to solve complex mathematical problems. Recently, Yang et al. [34] proposed a high dropout-resistant SA protocol based on HPRG and Paillier encryption. Nevertheless, centralized designs still face vulnerabilities to single points of failure.

As illustrated above, existing PPFL schemes have made the significant progress in addressing privacy concerns through different approaches. DP provides lightweight solutions but involves inherent tradeoffs between accuracy and privacy. HE ensures strong privacy guarantees but requires shared key management, and SA offers promising privacy preservation yet encounters efficiency challenges in dropout scenarios. Moreover, centralized designs not only remain vulnerable to single points of failure but also fail to provide sufficient protection for global model privacy.

B. Blockchain-Based FL

Blockchain, characterized by its decentralization, immutability, and transparency, has been widely adopted in various domains, including finance, supply chain, and healthcare [35], [36]. By smart contracts, blockchain enables automated transaction execution while ensuring data consistency and reliability through consensus mechanisms. The integration of blockchain with FL effectively addresses inherent security vulnerabilities in FL, such as the single point of failure risk of centralized servers [37]. Researchers have proposed various blockchain-based FL solutions in recent years. For example, Lu et al. [38] presented an asynchronous FL framework based on hybrid blockchain for secure data sharing in the Internet of Vehicles. Feng et al. [39] proposed a decentralized FL framework utilizing blockchain technology for model aggregation and cross-domain authentication of participants in FL. Han et al. [40] developed a blockchain-based FL architecture with a dynamic incentive mechanism to enhance client participation and ensure system security. However, these methods primarily focus on system security and model performance, with limited consideration of privacy issues. To address privacy concerns, Qi et al. [41] proposed a consortium blockchain-based FL framework with DP for secure traffic flow prediction. Ma et al. [42] introduced a BPFL framework that integrates pairwise additive masking and the Chinese remainder theorem (CRT), enabling collaborative learning while preserving participant privacy. However, it failed to effectively handle participant dropout scenarios. Fang et al. [43] proposed a blockchain-based FL framework with privacy-preserving and gradient verification mechanisms, which achieves decentralized training while handling client dropouts through verifiable secret sharing. However, their approaches incur the high communication overhead during mask reconstruction.

In summary, existing blockchain-based FL frameworks typically focus on enhancing security or model performance but face challenges in privacy protection and client dropout handling. In contrast, our proposed PBFL framework provides a comprehensive solution by integrating HE, single-masking, and blockchain, effectively addressing these limitations. Table I presents a detailed comparison between existing methods and our proposed PBFL in terms of technical approach, local model privacy protection, global model privacy protection, client dropout handling, high-efficiency, and blockchain integration.

III. PRELIMINARIES

A. Federated Learning

FL is a distributed ML paradigm that enables model training on decentralized data while preserving data privacy. We

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:04:08 UTC from IEEE Xplore. Restrictions apply.

consider an FL system consisting of a central server and n clients. Each client i possesses a local dataset $\mathcal{D}_i = \{(x_j, y_j) \mid j = 1, 2, \ldots, m\}$, where x_j and y_j represent the feature and label of the jth data sample, respectively. The size of the local dataset is denoted as $|\mathcal{D}_i|$. The training objective of FL is to minimize the global loss across all clients

$$\boldsymbol{w}^* = \arg\min_{\boldsymbol{w}} \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}_i(\boldsymbol{w}) \tag{1}$$

where $\boldsymbol{w}^*$ represents the optimal global model, and $\boldsymbol{w} = [w_1, w_2, \ldots, w_d]^T$ is the global model parameter vector. The local loss function $\mathcal{L}_i(\boldsymbol{w}_i^r)$ for each client i is defined as

$$\mathcal{L}_i\left(\boldsymbol{w}_i^r\right) = \frac{1}{|\mathcal{D}_i|} \sum_{j=1}^{|\mathcal{D}_i|} l\left(\boldsymbol{w}_i^r, x_j, y_j\right) \tag{2}$$

where the vector $\boldsymbol{w}_i^r = [w_{i,1}^r, w_{i,2}^r, \ldots, w_{i,d}^r]^T$ represents local model after the rth iteration, and $l(\boldsymbol{w}_i^r, x_j, y_j)$ denotes the loss for the jth data sample. For linear regression tasks, the loss function is defined as: $l(\boldsymbol{w}_i^r, x_j, y_j) = (1/2)(\boldsymbol{w}_i^{rT} x_j - y_j)^2$.

During the rth iteration, the server broadcasts the current global model $\boldsymbol{w}^{r-1}$ to clients. Each client i then updates its local model using stochastic gradient descent (SGD) algorithm [7]

$$\boldsymbol{w}_i^r = \boldsymbol{w}_i^r - \eta \nabla \mathcal{L}_i\left(\boldsymbol{w}_i^r\right) \tag{3}$$

where η is the learning rate. $\nabla \mathcal{L}_i(\boldsymbol{w}_i^r) = [(\partial \mathcal{L}_i/\partial w_{i,1}^r), (\partial \mathcal{L}_i/\partial w_{i,2}^r), \ldots, (\partial \mathcal{L}_i/\partial w_{i,d}^r)]^T$ denotes the gradient of $\mathcal{L}_i$ with respect to $\boldsymbol{w}_i^r$, where $(\partial \mathcal{L}_i/\partial w_{i,d}^r)$ is the gradient value of the dth element in $\boldsymbol{w}_i^r$.

Subsequently, clients send their updated local model $\boldsymbol{w}_i^r$ to the central server for aggregation

$$\boldsymbol{w}^r = \frac{1}{n} \sum_{i=1}^{n} \boldsymbol{w}_i^r. \tag{4}$$

Finally, the central server sends the updated global model $\boldsymbol{w}^r$ to all clients. This iterative process continues until the global model converges or meets a predefined stopping criterion.

B. Paillier Cryptosystem

The Paillier cryptosystem [44] is an additively HE scheme that has gained widespread attention in privacy-preserving applications. It consists of three key algorithms.

1) *$(pk, sk) \leftarrow$ Pai.KeyGen(k):* Given a security parameter k, two large prime numbers p and q are selected such that $|q| = |p| = k$. Calculate $N = p \cdot q$ and $\lambda = lcm(p-1, q-1)$. Define $L(x)$ as $L(x) = ([x-1]/N)$. Choose a generator $g \in \mathbb{Z}_{N^2}^*$ and calculate $\mu = (L(g^\lambda \bmod N^2))^{-1}$. The public key and private key pairs are $pk = (N, g)$ and $sk = (\lambda, \mu)$, respectively.
2) *$[\![M]\!] \leftarrow$ Pai.Enc(M, pk):* Given a message $M \in \mathbb{Z}_N$, the ciphertext can be calculated as $[\![M]\!] = g^M \cdot r^N \bmod N^2$, where $r \in \mathbb{Z}_N^*$ is a random number.
3) *$M \leftarrow$ Pai.Dec($[\![M]\!]$, sk):* Given the ciphertext $[\![M]\!]$, the plaintext is recovered as $M = L([\![M]\!]^\lambda \bmod N^2) \cdot \mu \bmod N$.

Moreover, the Paillier cryptosystem has additive homomorphic properties. For any $m_1, m_2 \in \mathbb{Z}_N$ under the same public key pk, we have

$$\begin{aligned} [\![m_1 + m_2]\!] &= [\![m_1]\!] \cdot [\![m_2]\!] \bmod N^2 \\ [\![cm]\!] &= [\![m]\!]^c \bmod N^2. \end{aligned} \tag{5}$$

C. Shamir Secret Sharing

In this article, we employ the (t, n)-threshold Shamir secret sharing scheme [45] to securely distribute a secret s into n distinct shares, where t represents the reconstruction threshold and $0 < t \leq n$. This cryptographic protocol ensures that any subset of t or more shares can reconstruct the secret s, while fewer than t shares reveal no information about s. The protocol operates as follows.

1) *Secret Sharing:* The secret holder selects $t-1$ random coefficients $a_1, \ldots, a_{t-1}$ from a finite field $\mathbb{Z}_p$ and constructs a polynomial $f(x)$ of degree $t-1$: $f(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{t-1} x^{t-1} \bmod p$, where $s = f(0)$ is the secret. The secret holder then evaluates $f(x)$ at n distinct, nonzero points to generate the shares: $y_i = f(x_i) \bmod p, i \in 1, 2, \ldots, n$. Subsequently, the shares are securely distributed to the n users, where each user receives a unique share (x_i, y_i).
2) *Secret Reconstruction:* To reconstruct the secret s, any subset of at least t shares $(x_1, y_1), (x_2, y_2), \ldots, (x_t, y_t)$ can be used. The reconstruction process employs Lagrange interpolation to recover the polynomial $f(x)$ and subsequently derive the secret: $s = a_0 = f(0) = \sum_{j=0}^{t} f(x_j) \prod_{m=0, m \neq j}^{t} (x_m / [x_m - x_j]) \bmod p$.

Moreover, Shamir secret sharing exhibits the property of additive homomorphism. Specifically, if two secrets s_1 and s_2 are shared using the same t-out-of-n scheme, the sum of their corresponding shares forms a valid sharing of the secret $s_1 + s_2$. Formally, if y_{j1} and y_{j2} are the shares of s_1 and s_2 for user j, then $y_j = (y_{j1} + y_{j2}) \bmod p$ is a share of the secret $s = (s_1 + s_2) \bmod p$.

D. Diffie–Hellman Key Agreement

Diffie–Hellman key agreement [46] is used to establish a shared secret key between two parties. The protocol consists of the following algorithms.

1) *$(\mathbb{G}, g, p') \leftarrow$ KA.Param(k):* Given the security parameter k, this algorithm generates the public parameters for the key agreement. Here, $\mathbb{G}$ is a cyclic group of large prime order p', g is a generator of $\mathbb{G}$.
2) *$(pk_i, sk_i) \leftarrow$ KA.KeyGen($\mathbb{G}$, g, p'):* The key generation algorithm outputs private key sk_i and public key $pk_i = g^{sk_i} \bmod p'$ for each user i.
3) *$sk_{i,j} \leftarrow$ KA.KeyAgree(sk_i, g^{sk_j}):* The key agreement algorithm takes a private key sk_i and the other party's public key g^{sk_j} as inputs, and outputs the agreement key $sk_{i,j}$. In practical applications, $sk_{i,j}$ is typically calculated as $(H(g^{sk_j})^{sk_i})$ for convenience.

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:04:08 UTC from IEEE Xplore. Restrictions apply.

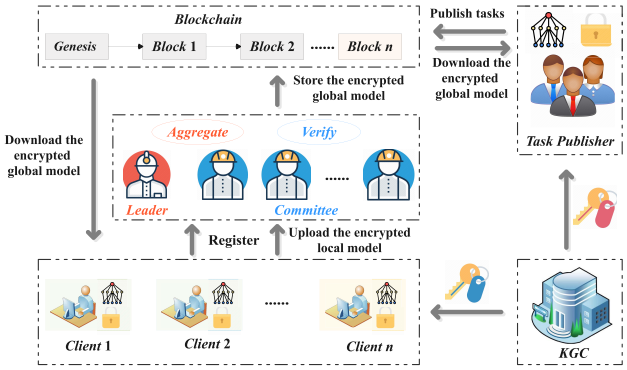


Fig. 1. Architecture of the proposed PBFL.

E. Homomorphic Pseudorandom Generator

A pseudorandom generator (PRG) [47] is fundamental cryptographic primitive designed to create a sequence of approximate random numbers from a random seed. A secure PRG can be constructed by pseudorandom function (PRF) in the random oracle model [48]. A secure PRF is represented by the function $F : \mathcal{K} \times \mathcal{X} \rightarrow \mathcal{Y}$, where $\mathcal{K}$ is group. For any $k_1, k_2 \in \mathcal{K}$, if F is homomorphic with respect to the key, there exists an efficient procedure to calculate $F(k_1 \oplus k_2, x)$ using $F(k_1, x)$ and $F(k_2, x)$. The key homomorphic PRF can be constructed as follows.

Let $\mathbb{G}'$ be a group of prime order q', and $H : \mathcal{X} \rightarrow \mathbb{G}'$ be a hash function modeled as a random oracle. Define the function $F : \mathbb{Z}_{q'} \times \mathcal{X} \rightarrow \mathbb{G}'$ by

$$F(s, x) = H(x)^s. \tag{6}$$

For $s_1, s_2 \in \mathbb{Z}_{q'}$, it holds that $F(s_1 + s_2, x) = F(s_1, x) \cdot F(s_2, x)$. Under the decision Diffie–Hellman (DDH) assumption in $\mathbb{G}'$, F is proven to be a secure PRF. Therefore, a seed homomorphic PRG HPRG(s) can be constructed directly by F. For instance, considering an HPRG(s) with public parameter vector $\boldsymbol{x} = [x_1, x_2, \ldots, x_m]$, it generates a sequence of elements

$$\text{HPRG}(s) = [F(s, x_1), F(s, x_2), \ldots, F(s, x_m)]. \tag{7}$$

The security of the HPRG is based on the indistinguishability of its output from uniformly sampled elements, provided that the seed s remains secret. This ensures that an adversary cannot distinguish the HPRG output from true random values. In this article, $\mathbb{Z}_{q'}^{\times}$, the multiplicative group of integers modulo a large safe prime q, is chosen as the finite cyclic group $\mathbb{G}$. This selection supports the DDH assumption, thus fortifying the HPRG's security [49].

IV. System Model and Threat Model

In this section, we will describe the system model, threat model, and design goals, respectively.

A. System Model

Fig. 1 illustrates our proposed PBFL architecture, which includes five key components: 1) a trusted key generation center (KGC); 2) task publisher (TP); 3) clients; 4) the blockchain system; and 5) validation committee. The details of each component are elaborated as follows.

1) *KGC:* An independent, trusted entity responsible for providing essential public and private key management services to all entities involved.
2) *TP:* An entity that defines learning objectives, datasets, and initial model parameters, and broadcasts this information to the blockchain.
3) *Clients:* A set of intelligent IIoT devices, denoted as $\mathcal{U} = \{1, 2, \ldots, n\}$, where n is the total number of clients and u_i represents the ith client. Each client possesses local datasets and computing capabilities, trains a local model, and periodically submits masked local model updates to the blockchain as transactions.
4) *Validation Committee:* A group of selected nodes that verify the integrity and validity of the masked local model before they are added to the blockchain. This committee employs the Algorand consensus protocol to ensure that only valid models are aggregated, thereby maintaining the accuracy and reliability of the aggregated model.
5) *Blockchain:* The blockchain serves as a decentralized and tamper-resistant ledger that maintains two critical types of information: a) encrypted global model parameters and b) masked local model updates. Each block contains multiple transactions that have been validated through consensus mechanism. When a new block is appended to the chain, all participants can access the latest model updates. This mechanism ensures the transparency and traceability of the entire training process while maintaining data integrity.

B. Threat Model

In this article, we assume that the KGC is a trusted third party and does not collude with other entities. The clients are considered honest-but-curious. They follow the FL protocol by performing local training on their datasets and submitting correct local models with the mask. However, they may attempt to infer additional information from other clients' local models. We assume that the validation committee is not completely trustworthy and there may be compromised nodes among its members that try to manipulate the consensus process. In addition, these compromised nodes may collude with clients to obtain private information. The blockchain itself is assumed to be secure against common attacks, such as 51% attacks, due to its decentralized nature and the security properties of the underlying consensus mechanism.

C. Design Goals

Based on the aforementioned threat model, the primary objective of our work is to design a privacy-preserving and efficient model aggregation protocol for the procedure of FL training. Our specific goals are as follows.

1) *Privacy:* Our proposed scheme aims to protect clients' data from disclosure. No curious entity is able to access or infer the original information of the client.

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:04:08 UTC from IEEE Xplore. Restrictions apply.

TABLE II
NOTATION DEFINITIONS

Symbol	Definition
g	Generator of group $\mathbb{G}$
$\mathbb{Z}_p$	Finite field of order p
d_i^{sk}/d_i^{pk}	The client's signing public and private keys
$(\lambda, \mu)/(N, g)$	Paillier public and private keys of the TP
$\mathcal{U}$	Set of all initially registered clients
$\mathcal{U}_1$	Set of clients who complete the secret sharing phase
$\mathcal{U}_2$	Set of clients who upload their local models
$\mathcal{D}_i$	Local dataset of client u_i
$\mathcal{T}$	Training cycle
$\boldsymbol{w}_i^r$	Local model of client u_i in round r
$\boldsymbol{w}^r$	Global model in round r
$\nabla\mathcal{L}_i\left(\boldsymbol{w}_i^r\right)/\mathbf{g}_i^r$	Local model gradient of client u_i in round r
$[\![\boldsymbol{w}_i^r]\!]$	Encrypted local model of client u_i in round r
$[\![\mathbf{g}_i^r]\!]$	Encrypted local model gradient of client u_i in round r
$[\![\mathrm{g}_{i,k}^r]\!]$	The k-th element of the encrypted local model gradient of client u_i in round r
$[\![\boldsymbol{w}^r]\!]$	Encrypted global model in round r
s_i	Secret share of client u_i
$TX_{i,r}$	Model upload transaction of client u_i in round r
σ	Signature for model upload transaction

2) *Robustness:* The proposed scheme should ensure that random mask can be offset when aggregating all local gradients, even if some clients drop out during the training process.
3) *Efficiency:* The proposed scheme should be efficient, with low computational and communication overhead.
4) *Auditability:* The behavior of each entity during the training should be auditable. This includes the verification of local model updates and aggregation, ensuring that any illegal behavior is traceable.

V. OUR SCHEME

In this section, we present the PBFL framework, which consists of the following four phases: 1) system initialization; 2) local model training and uploading; 3) global model aggregation; and 4) model updating. For clarity, the notation used in this article is shown in Table II.

A. System Initialization

1) System Parameter Generation: The KGC is responsible for generating the system parameters. Upon the publication of a new training task, the KGC selects a system security parameter k and runs Pai.KeyGen(k) of the Paillier HE algorithm to generate a public key $pk = (N, g)$ and a corresponding private key $sk = (\lambda, \mu)$. The KGC then securely transmits (λ, μ) to the TP via a secure channel for confidential storage.

2) User Registration: Before participating in FL training, client i must register with the system. After receiving a registration request, the KGC generates a signature key pair (d_i^{sk}, d_i^{pk}) for the client. The KGC securely transmits the private key d_i^{sk} to the client through a secure channel and publishes the public key d_i^{pk} bound to the client's identity u_i.

3) Key Agreement: After registration, each client generates a Diffie–Hellman key pair (sk_i, pk_i) and securely distributes its public key pk_i. Subsequently, any two clients u_i and u_j use the Diffie–Hellman protocol to compute a unique shared key $sk_{i,j} = \mathrm{KA.KeyAgree}(sk_i, pk_j)$. These shared keys are used for authenticated encryption in the subsequent Shamir secret sharing process, ensuring secure data exchange in FL.

4) Model Initialization: The TP first specifies the training task parameters, including the initial global model $\boldsymbol{w}^0$, learning rate η, training batch size B, and reward R. Then, the TP runs $\mathrm{Pai.Enc}(\boldsymbol{w}^0, pk)$ of the Paillier HE algorithm to encrypt $\boldsymbol{w}^0$ and generate encrypted global model $[\![\boldsymbol{w}^0]\!]$. Next, the TP publishes the task information $\{[\![\boldsymbol{w}^0]\!], \eta, B, R\}$ on the blockchain via a smart contract in the form of transactions. The TP is also required to deposit a predefined amount of money as a guarantee of their commitment to the task. Finally, clients can retrieve the task details from blockchain and download the encrypted global model $[\![\boldsymbol{w}^0]\!]$ for local training. Clients participating in training tasks are also required to pay a deposit of a certain amount. Upon successful completion of the tasks, the deposits will be refunded, and the clients will be rewarded based on their contributions.

B. Local Model Training and Uploading

In the rth iteration, clients download the encrypted global model $[\![\boldsymbol{w}^{r-1}]\!]$ from the blockchain and utilize it to update their local models. Since the global model is encrypted, the clients execute the SGD algorithm within the encrypted domain. This secure update process is achieved by the additively homomorphic properties of Paillier encryption. The detailed description of local model training in the ciphertext domain can be found in Algorithm 1.

1) Local Model Training in Ciphertext State: For ease of representation, the gradient $\nabla\mathcal{L}_i(\boldsymbol{w}_i^r)$ is denoted as $\mathbf{g}_i^r = [\mathrm{g}_{i,1}^r, \mathrm{g}_{i,2}^r, \ldots, \mathrm{g}_{i,d}^r]^T$. Thus, the encrypted local gradient can be represented as

$$[\![\mathbf{g}_i^r]\!] = \left[[\![\mathrm{g}_{i,1}^r]\!], [\![\mathrm{g}_{i,2}^r]\!], \ldots, [\![\mathrm{g}_{i,d}^r]\!]\right]^T. \tag{8}$$

For any element $g_{i,k}^r$ in $\mathbf{g}_i^r$, the ciphertext can be computed using the Paillier algorithm as follows:

$$\begin{aligned}[\![\mathbf{g}_{i,k}^r]\!] &= \left[\!\!\left[\frac{\partial\mathcal{L}_i\left(\boldsymbol{w}_i^r\right)}{\partial w_{i,k}^r}\right]\!\!\right] = \left[\!\!\left[\frac{1}{|\mathcal{D}_i|}\sum_{j=1}^{|\mathcal{D}_i|}\mathrm{g}_{i,j,k}^r\right]\!\!\right] \\ &= \left(\prod_{j\in\mathcal{D}_i}[\![\mathrm{g}_{i,j,k}^r]\!]\right)^{|\mathcal{D}_i|^{-1}} \left(\mathrm{mod}N^2\right)\end{aligned} \tag{9}$$

where $\mathrm{g}_{i,j,k}^r$ denotes the gradient of the kth element for the jth sample in the local dataset $\mathcal{D}_i$. We further derive as follows:

$$\begin{aligned}[\![\mathrm{g}_{i,j,k}^r]\!] &= \left[\!\!\left[\frac{\partial l\left(\boldsymbol{w}_i^r, x_j, y_j\right)}{\partial w_{i,k}^r}\right]\!\!\right] \\ &= [\![\left(\boldsymbol{w}_i^{rT}x_j - y_j\right)x_{j,k}]\!] \\ &= [\![y_j]\!]^{-x_{j,k}}\left(\prod_{m=1}^{d}[\![w_{i,m}^r]\!]^{x_{j,m}}\right)^{x_{j,k}} \left(\mathrm{mod}N^2\right)\end{aligned} \tag{10}$$

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:04:08 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Local Model Training in the Ciphertext Domain

Input : Global model $[\![\boldsymbol{w}^{r-1}]\!]$, local dataset $\mathcal{D}_i$
Output: $[\![\widetilde{\boldsymbol{w}}_i^r]\!]$, s_{sum}^i
Initialization: $[\![\boldsymbol{w}_i^r]\!] = [\![\boldsymbol{w}^{r-1}]\!]$;
/* Update the local model */
for *each client* $u_i \in \mathcal{U}$ **do**
  for *each element* $[\![g_{i,k}^r]\!]$ *in* $[\![\mathbf{g}_i^r]\!]$ **do**
    $[\![g_{i,k}^r]\!] \leftarrow \left(\prod_{j\in\mathcal{D}_i}\left[\!\left[\mathrm{g}_{i,j,k}^r\right]\!\right]\right)^{|\mathcal{D}_i|^{-1}} \left(\mathrm{mod}N^2\right)$
  end
  $[\![\boldsymbol{w}_i^r]\!] \leftarrow [\![\boldsymbol{w}^{r-1}]\!][\![\mathbf{g}_i^r]\!]^{-\eta}\left(\mathrm{mod}N^2\right)$
end
/* Masking the Encrypted Local Model */
for *each client* $u_i \in \mathcal{U}$ **do**
  Select a random seed s_i;
  Generate mask vector $\boldsymbol{c}_i \leftarrow HPRG(s_i)$;
  Obfuscate encrypted local model $[\![\widetilde{\boldsymbol{w}}_i^r]\!] \leftarrow [\![\boldsymbol{w}_i^r]\!] \cdot HPRG(s_i)$;
end
/* Shamir-Based Seed Sharing */
for *each client* u_i **do**
  Randomly select polynomial $f_i(x)$;
  for *each* $u_j \in \mathcal{U}$ **do**
    Compute and distribute share $(u_j, f_i(u_j))$;
  end
  Collect shares from $\mathcal{U}_1$;
  if $\phi_{train} = |\mathcal{U}_1| \geq t$ **then**
    $s_{sum}^i \leftarrow \sum_{u_j\in\mathcal{U}_1} f_j(u_i) \cdot \prod_{\substack{u_m\in\mathcal{U}_1\\ m\neq j}} (\frac{u_i-u_m}{u_j-u_m})$;
  end
end
/* Uploading the Masked Encrypted Model */
for *each client* u_i **do**
  Create transaction:
  $TX_{i,r} \leftarrow \left\{r, ID_i, [\![\widetilde{\boldsymbol{w}}_i^r]\!], \sigma\left(d_i^{sk}, H\left([\![\widetilde{\boldsymbol{w}}_i^r]\!] \| s_{\text{sum}}^i\right)\right), s_{\text{sum}}^i\right\}$;
end

where $x_{j,k}$ is the kth element of the feature vector x_j. After computing the encrypted local gradients, client u_i updates the local model in the encrypted domain, as follows:

$$\begin{aligned}[\![\boldsymbol{w}_i^r]\!] &= \left[\!\left[\boldsymbol{w}^{r-1} - \eta\nabla\mathcal{L}_i\left(\boldsymbol{w}_i^r\right)\right]\!\right] \\ &= \left[\!\left[\boldsymbol{w}^{r-1}\right]\!\right][\![\mathbf{g}_i^r]\!]^{-\eta}\left(\mathrm{mod}N^2\right). \end{aligned} \tag{11}$$

2) Masking the Encrypted Local Model: Since the TP holds the private key sk for HE, they can decrypt the local models once uploaded to the blockchain. To enhance privacy protection, inspired by [50], we apply a single mask technique before submitting these local models to the blockchain. Specifically, each client u_i first selects a random seeds s_i and use the $HPRG(\cdot)$ to generate a mask vector $\boldsymbol{c}_i$ as follows:

$$\begin{aligned}\boldsymbol{c}_i &= HPRG(s_i) \\ &= [F(s_i,1), F(s_i,2), \ldots, F(s_i,d)] \\ &= \left[H(1)^{s_i}, H(2)^{s_i}, \ldots, H(d)^{s_i}\right]. \end{aligned} \tag{12}$$

Then, client u_i uses the mask vector to obfuscate the encrypted local model. The masked encrypted model can be represented as

$$\begin{aligned}[\![\widetilde{\boldsymbol{w}}_i^r]\!] &= [\![\boldsymbol{w}_i^r]\!] \cdot \mathrm{HPRG}(s_i) \\ &= \mathrm{Pai.Enc}\left(\boldsymbol{w}_i^r, pk\right) \cdot \mathrm{HPRG}(s_i) \quad (\mathrm{mod}q) \\ &= \Big[g^{w_{i,1}^r} r_i^N H(1)^{s_i}, g^{w_{i,2}^r} r_i^N H(2)^{s_i}, \ldots, \\ &\quad g^{w_{i,d}^r} r_i^N H(d)^{s_i}\Big]. \end{aligned} \tag{13}$$

3) Shamir-Based Seed Sharing: To handle the issue of clients dropping out during the training, we use the Shamir secret sharing method that allows the seeds s_i to be shared among the clients. This ensures that even if clients drop out, the mask can still be removed during the aggregation phase. Client u_i randomly selects a polynomial $f_i(x)$ over $\mathbb{Z}_p$ with the constant term $f_i(0)$ as the secret s_i

$$f_i(x) = s_i + a_1 x + a_2 x^2 + \cdots + a_{(t-1)} x^{t-1} (\mathrm{mod}p). \tag{14}$$

Next, client u_i computes the polynomial $f_i(x)$ at n distinct points and distributes these shares $\{(u_j, f_i(u_j)) | u_j \in \mathcal{U}\}$ to other user u_j using authenticated encryption. Meanwhile, the other $n-1$ users will distribute their shares to client u_i. When client u_i collects $\phi_{\text{train}} = |\mathcal{U}_1| \leq n$ shares from a set $\mathcal{U}_1$, where $\mathcal{U}_1$ denotes the set of clients who successfully complete the secret sharing phase. If $\phi_{\text{train}} \geq t$, a single share s_{sum}^i of the overall sum $s_{\text{sum}} = \sum_{u_i\in\mathcal{U}_1} s_i$ can be reconstruction from the Lagrange interpolation

$$s_{\text{sum}}^i = f_{\text{sum}}(u_i) = \sum_{u_j\in\mathcal{U}_1} f_j(u_i) \cdot \prod_{\substack{u_m\in\mathcal{U}_1\\ m\neq j}} \frac{u_i - u_m}{u_j - u_m}. \tag{15}$$

4) Uploading the Masked Encrypted Model: The client u_i uploads its masked encrypted local model $[\![\widetilde{\boldsymbol{w}}_i^r]\!]$ to the blockchain. Each transaction sent by the client contains the following information:

$$TX_{i,r} = \left\{r, \mathrm{ID}_i, [\![\widetilde{\boldsymbol{w}}_i^r]\!], \sigma\left(d_i^{sk}, H\left([\![\widetilde{\boldsymbol{w}}_i^r]\!] || s_{\text{sum}}^i\right)\right), s_{\text{sum}}^i\right\}$$

where $\sigma(d_i^{sk}, H([\![\widetilde{w}_i^r]\!] || s_{\text{sum}}^i))$ is the transaction signature generated by client i, r denotes the current iteration number, and ID_i is the identifier of the client u_i.

C. Global Model Aggregation

At the end of the training cycle $\mathcal{T}$, the validation committee is responsible for verifying the validity of the transactions. Subsequently, a leader and a set of validators are selected through the Algorand consensus algorithm to carry out the global model aggregation and verification. The final aggregated model is then stored to the blockchain. The detail of the process is described as follows.

1) Transaction Legitimacy Verification: The validation committee check whether all clients have finished model uploading task before the training cycle ends. If some clients fail to upload their models, the penalty mechanism will be triggered to deduct their deposits. Meanwhile, the validation committee verifies the transaction's signature to confirm its authenticity and integrity. The validated transactions will be

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:04:08 UTC from IEEE Xplore. Restrictions apply.

added to the transaction pool and wait to be processed. The set $\mathcal{U}2$ indicates clients who submitted their models on time and successfully passed verification.

2) Aggregation and Unmasking: The leader executes transactions in the pool to aggregate the encrypted local models submitted by clients, following the process below:

$$\begin{aligned}\sum_{u_i\in\mathcal{U}_2}[\![\widetilde{\boldsymbol{w}}_i^r]\!] &= \prod_{u_i\in\mathcal{U}_2}\text{Pai.Enc}\big(\boldsymbol{w}_i^r, pk\big)\\ &\quad\cdot\prod_{u_i\in\mathcal{U}_2}\text{HPRG}(s_i)\pmod q\\ &= \text{PHE.Enc}\left(pk,\sum_{u_i\in\mathcal{U}_2}\boldsymbol{w}_i^r\right)\\ &\quad\cdot\text{HPRG}\left(\sum_{u_i\in\mathcal{U}_2}s_i\right)(\text{mod}q).\end{aligned}\tag{16}$$

If $|\mathcal{U}_2|\geq t$, it indicates that the leader has successfully received sufficient shares from at least t distinct clients. By leveraging the additive homomorphic property of the t-out-of-n Shamir secret sharing scheme, s_{sum} can be reconstructed as

$$s_{\text{sum}} = f_{\text{sum}}(0) = \sum_{u_i\in\mathcal{U}_2}s_{sum}^i\cdot\prod_{\substack{u_m\in\mathcal{U}_2\\ m\neq j}}\frac{-u_m}{u_j-u_m}.\tag{17}$$

Once s_{sum} is recovered, the leader can unmask the aggregated global model

$$\begin{aligned}[\![\boldsymbol{w}^r]\!] &= \sum_{u_i\in\mathcal{U}_2}[\![\widetilde{\boldsymbol{w}}_i^r]\!]/\,\text{HPRG}\left(\sum_i^{|\mathcal{U}_2|}s_i\right)\pmod q\\ &= \sum_{u_i\in\mathcal{U}_2}[\![\widetilde{\boldsymbol{w}}_i^r]\!]/\,\text{HPRG}(s_{\text{sum}})\pmod q\end{aligned}\tag{18}$$

where $[\![\boldsymbol{w}^r]\!]$ is the aggregated model in ciphertext state for the rth round.

3) Block Generation and Broadcast: After completing the aggregation operation, the leader creates a new transaction $TX_{Agg}=\{r,[\![\boldsymbol{w}^r]\!],\sigma(sk_i,H([\![\boldsymbol{w}^r]\!]))\}$. The leader then collapses all transactions into a new block, denoted as $NewBlock=\{TX_{Agg},TX_{1,r},TX_{2,r},\ldots,TX_{n,r}\}$, and broadcasts this block to the nodes within the validation committee for verification. If the block is validated by the majority of the validation committee members, the new block will be appended to the blockchain. If the verification fails, the block will be rejected and not added to the blockchain.

D. Update Phase

In this phase, each client u_i downloads the last encrypted global model $[\![\boldsymbol{w}^r]\!]$ from the blockchain and uses it to update their local models, initiating a new training round. This process is repeated iteratively until the model meets the predefined performance requirements. After the task is completed, TP downloads the final encrypted global model from the blockchain and decrypts it using its private key. The smart contract automatically distributes the TP's deposit as a reward to all participating clients, incentivizing them to continue participating in future training tasks.

Algorithm 2: Global Model Aggregation

Input : The clients identifier ID_i, training cycle $\mathcal{T}$, current cycle $\mathcal{T}_{current}$, $\mathcal{U}_2$
Output: $[\![\boldsymbol{w}^t]\!]$

Initialization: The set of online clients $\mathcal{U}_2$;
if $\mathcal{T}_{current}>\mathcal{T}$ **then**
  for $u_i\in\mathcal{U}$ **do**
    /* Retrieve transaction and validity */
    $txHash, boolVal = GetTxByID\ (ID_i)$;
    $boolValid =$ VerifySignature($txHash$);
    if *boolVal* ***and*** *boolValid* **then**
      $U_2 = U_2.append(ID_i)$;
    end
  end
end
for $U_i\notin\mathcal{U}_2$ **do**
  $boolVal = DeductDeposit(ID_i)$;
end
if $|\mathcal{U}_2|\geq t$ **then**
  $\sum_{u_i\in\mathcal{U}_2}[\![\widetilde{\boldsymbol{w}}_i^r]\!] = \prod_{u_i\in\mathcal{U}_2}\text{Pai. Enc}\big(\boldsymbol{w}_i^r,pk\big)\cdot\prod_{u_i\in\mathcal{U}_2}\text{HPRG}(s_i)\pmod q$
  Reconstruct s_{sum} using Shamir's Secret Sharing: $s_{\text{sum}} = f_{\text{sum}}(0) = \sum_{u_i\in\mathcal{U}_2}s_{\text{sum}}^i\cdot\prod_{\substack{u_m\in\mathcal{U}_2\\ m\neq j}}(\frac{-u_m}{u_j-u_m})$;
  Unmask the aggregated model: $[\![\boldsymbol{w}^t]\!] = \sum_{u_i\in\mathcal{U}_2}[\![\widetilde{\boldsymbol{w}}_i^t]\!]/\,\text{HPRG}(s_{\text{sum}})\pmod q$;
end
Create new transaction TX_{Agg} and consolidate all transactions into a new block *NewBlock*;
Broadcast *NewBlock* to validation committee for verification;
if *block validated by majority* **then**
  Append *NewBlock* to the blockchain;
else
  Reject *NewBlock*;
end

VI. Security Analysis

In this section, we analyze the proposed scheme in terms of both privacy and security.

A. Privacy Analysis

We prove that our protocol provides strong privacy guarantees, ensuring that clients' local models remains protected from disclosure. To facilitate this proof, we first introduce some key notations. Let $\mathcal{U}$ denote the set of all initially registered clients, $\mathcal{U}_1$ represent the set of clients who successfully complete the secret sharing phase, and $\mathcal{U}_2$ indicate the set of clients who have completed the upload of their local models. These sets satisfy the relationship $\mathcal{U}_2\subseteq\mathcal{U}_1\subseteq\mathcal{U}$. For each client u_i, w_i denotes its local model, and $\boldsymbol{w}_{\mathcal{U}_r}=\{\boldsymbol{w}_i\}_{i\in\mathcal{U}_r}$ represents the gradient set for any $\mathcal{U}_r\subseteq\mathcal{U}$.

In our scheme, the view of a party consists of its inputs, randomness, and all messages received from other parties. Once a party exits the protocol, its view remains fixed at the state

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:04:08 UTC from IEEE Xplore. Restrictions apply.

corresponding to the last received message. For the sake of description, we use the symbol B to represent the leader node and introduce the random variable $\mathbf{REAL}_{\mathcal{W}}^{\mathcal{U},t,k}(\boldsymbol{w}_{\mathcal{U}}, \mathcal{U}_1, \mathcal{U}_2)$ to represent the joint view of all parties within the set $\mathcal{W} \subseteq \mathcal{U} \cup B$. The parameters t and k indicate the threshold and security parameter used in our protocol.

We analyze the privacy guarantees of our protocol under two scenarios: 1) honest but curious clients with an honest leader and 2) honest but curious clients and leader. Next, we present Theorem 1, which demonstrates that any collusion involving less than t clients, excluding the leader, is insufficient to compromising the private information of other clients.

Theorem 1 (Privacy Protection Against Honest-But-Curious Clients With an Honest Leader): For all $t, k, \mathcal{W} \subseteq \mathcal{U}, \boldsymbol{w}_{\mathcal{U}}, \mathcal{U}$ with $|\mathcal{U}| \geq t$, and $\mathcal{U}_2 \subseteq \mathcal{U}_1 \subseteq \mathcal{U}$, there is a probabilistic polynomial-time (PPT) simulator, denoted as **SIM**, whose output is indistinguishable from the output of $\mathbf{REAL}_{\mathcal{W}}^{\mathcal{U},t,k}$

$$\mathbf{REAL}_{\mathcal{W}}^{\mathcal{U},t,k}(\boldsymbol{w}_{\mathcal{U}}, \mathcal{U}_1, \mathcal{U}_2) \equiv \mathbf{SIM}_{\mathcal{W}}^{\mathcal{U},t,k}(\boldsymbol{w}_{\mathcal{W}}, \mathcal{U}_1, \mathcal{U}_2). \tag{19}$$

Proof: Given that the leader is honest, the joint view of clients in $\mathcal{W}$ remains independent of the inputs from other clients in $\mathcal{U}\backslash\mathcal{W}$. Hence, the simulator **SIM** can perfectly simulate the protocol by using the real inputs of honest but curious clients while replacing the inputs of honest clients in $\mathcal{U}\backslash\mathcal{W}$ with fabricated data (e.g., random numbers). It means that the simulated view of clients in $\mathcal{W}$ is indistinguishable from the actual view $\mathbf{REAL}_{\mathcal{W}}^{\mathcal{U},t,k}$. ■

Theorem 2 (Privacy Protection Against Honest-But-Curious Clients and the Leader): For all $t, k, \boldsymbol{w}_{\mathcal{U}}, \mathcal{W} \subseteq \mathcal{U} \cup B$, $|\mathcal{W}\backslash B| < t$, $\mathcal{U}$ with $|\mathcal{U}| \geq t$, and $\mathcal{U}_2 \subseteq \mathcal{U}_1 \subseteq \mathcal{U}$, there exists a PPT simulator **SIM** whose output is indistinguishable from the output of $\mathbf{REAL}_{\mathcal{W}}^{\mathcal{U},t,k}$

$$\mathbf{REAL}_{\mathcal{W}}^{\mathcal{U},t,k}(\boldsymbol{w}_{\mathcal{U}}, \mathcal{U}_1, \mathcal{U}_2) \equiv \mathbf{SIM}_{\mathcal{W}}^{\mathcal{U},t,k}(\boldsymbol{w}_{\mathcal{W}}, \mathcal{U}_1, \mathcal{U}_2) \tag{20}$$

where

$$\xi = \begin{cases} \sum_{u \in \mathcal{U}_2 \backslash \mathcal{W}} \boldsymbol{w}_u, & \text{if } |\mathcal{U}_2| \geq t \\ \perp, & \text{otherwise.} \end{cases}$$

Proof: We employ a standard hybrid argument to prove Theorem 2. The simulator **SIM** executes a series of sequential modifications on the random variable **REAL**, which ultimately makes the simulated view indistinguishable from the real view. In our hybrid argument, $(\mathbf{Hyb}_i)$, where $(i \in 0, 1, \ldots, 5)$, represents a secure modification to the original protocol. We define the hybrids as follows.

Hyb_0: This hybrid represents the real execution of the protocol, where all parties follow the prescribed steps without any modifications.

Hyb_1: In this hybrid, we modify the key generation process for honest clients $u_i \in \mathcal{U} \setminus \mathcal{W}$. The simulator replaces the output of KA.Agree(pk_j, sk_i) with a uniformly random encryption key $k_{i,j}$ for communication between u_i and each u_j in $\mathcal{U}\backslash\mathcal{W}$. The DDH assumption guarantees that this hybrid is computationally indistinguishable from Hyb_0.

Hyb_2: In this hybrid, we substitute ciphertexts of actual shares s_i^j of honest parties $u_i \in \mathcal{U} \setminus \mathcal{W}$ with encryptions of random values (e.g., 0, with appropriate length) in transmissions to other honest participants. The IND-CPA security property of the symmetric encryption scheme ensures that this hybrid remains indistinguishable from H_1.

Hyb_3: In this hybrid, for each honest client $u_i \in \mathcal{U}\backslash\mathcal{W}$, we replace s_i^j, i.e., the shares of honest clients u_i's seeds s_i, with randomly selected elements from the corresponding field. These modified shares are then distributed to the corrupted parties in $\mathcal{W}$. The security properties of Shamir's secret sharing scheme ensure that parties in $\mathcal{W}$ gain no information about the original seeds, as they possess fewer than t shares ($|\mathcal{W}| < t$). Therefore, the threshold property of the scheme maintains the confidentiality of the seeds, rendering $\mathbf{Hyb}_3$ computationally indistinguishable from $\mathbf{Hyb}_2$.

Hyb_4: In this hybrid, instead of computing the mask $\boldsymbol{r}_i$ using HPRG, we substitute $\boldsymbol{r}_i$ of each honest party with a randomly selected element of appropriate length from the finite cyclic group. The security of HPRG, which relies on the DDH assumption, ensures that this hybrid remains computationally indistinguishable from the previous one.

Hyb_5: In this hybrid, for the honest clients $u_i \in \mathcal{U}_2\backslash\mathcal{W}$, we sample $[\![\boldsymbol{x}_i]\!]$ uniformly at random, subject to $\sum_{\mathcal{U}_2\backslash\mathcal{W}}[\![\boldsymbol{x}_i]\!] = \sum_{\mathcal{U}_2\backslash\mathcal{W}}[\![\boldsymbol{w}_i]\!] = z$. We replace $[\![\widetilde{\boldsymbol{w}}_i]\!] = [\![\boldsymbol{w}_i]\!] \cdot \boldsymbol{r}_i$ with $[\![\widetilde{\boldsymbol{w}}_i]\!] = [\![\boldsymbol{x}_i]\!] \cdot \boldsymbol{r}_i$. Due to the uniform randomness of $\boldsymbol{r}_i$, the distribution of $[\![\boldsymbol{w}_i]\!] \cdot \boldsymbol{r}_i$ is computationally indistinguishable from that of $[\![\boldsymbol{x}_i]\!] \cdot \boldsymbol{r}_i$. Therefore, this hybrid remains indistinguishable from $\mathbf{Hyb}_4$.

By leveraging the properties established in the previous hybrids, we demonstrate that the combined view of the corrupted parties $\mathcal{W}$ in the simulation produced by **SIM** is computationally indistinguishable from their view in the real execution **REAL**. Consequently, we conclude that the protocol is secure against semi-honest adversaries, completing the security proof. ■

Theorem 3 (Global Model Privacy): If the Paillier homomorphic cryptosystem is secure, no entity other than the TP can decrypt the encrypted aggregation model.

Proof: In the task release phase, the TP encrypts the initial global model $\boldsymbol{w}^0$ using the Paillier homomorphic cryptosystem and sends the ciphertext $[\![\boldsymbol{w}^0]\!]$ to the blockchain. In the tth iteration, client u_i downloads the encrypted global model $[\![\boldsymbol{w}^{t-1}]\!]$ from the blockchain, perform local training in the encrypted domain, and upload $[\![\boldsymbol{w}_i^t]\!]$ to the blockchain. The leader and validators collect these encrypted local model updates and aggregate them to obtain $[\![\boldsymbol{w}^t]\!]$. Due to the semantic security of the Paillier cryptosystem, no entity without the private key can decrypt the ciphertexts throughout the training process. Thus, only the TP, who holds the homomorphic private key, can decrypt the aggregated ciphertext, ensuring the privacy of the global model. ■

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:04:08 UTC from IEEE Xplore. Restrictions apply.

B. Correctness of the Verification

Theorem 4: In our scheme, the leader is unable to forge a valid aggregation result that can pass the aggregation verification of the validation committee.

Proof: In the proposed scheme, the leader node elected by the Algorand consensus algorithm is responsible for collecting all encrypted model updates submitted by clients. The leader node uses these updates to perform model aggregation according to the predefined protocol and generates a new block, which includes the aggregated model parameters, a timestamp, and the leader's cryptographic proof, demonstrating its legitimacy in the election process. The leader broadcasts this block to all validators. Upon receiving the block, the validators first verify the leader's cryptographic proof to ensure the leader was legitimately elected. They also check the basic structure and content of the block to ensure it meets the protocol requirements. After verifying the block's validity, the validators vote on the block and broadcast their voting results. If the block proposal receives more than 2/3 of the support votes, the block is officially confirmed and added to the blockchain. Thus, in our mechanism, the verifiability of the aggregation results is achieved by utilizing the blockchain and consensus algorithm. ■

VII. Performance Evaluation

In this section, we conduct numerical experiments using real-world datasets to evaluate the performance of the proposed scheme.

A. Experimental Setting

1) Experiment Environment: The proposed mechanism was implemented and evaluated on a high-performance workstation running Ubuntu 20.04 LTS, equipped with an Intel i9 CPU, 64 GB of memory, and four NVDIA RTX 4090 GPUs. The FL framework was developed using PyTorch (version 1.6.0)[1] and Python (version 3.8.10).[2] A consortium blockchain was chosen as the platform to support decentralized and secure interactions. For cryptographic primitives, we employed the Paillier cryptosystem for HE with a 2048-bit key size, DDH-based HPRG with an SHA-256 hash to generate random masks, and Shamir's secret sharing scheme to handle client dropouts.

2) Datasets and Model: We selected the California Housing[3] to evaluate the performance of our proposed mechanism on regression tasks. The California Housing dataset consists of 20 640 samples, each containing eight features, used to predict the median house price. The dataset were randomly split into training (80%) and testing (20%) sets. To simulate FL scenarios, we assumed that there are total 200 local clients in the system, with each client receiving a randomly allocated subset of the data. For client-side training, we implemented a linear regression model with the following hyperparameter configurations: a learning rate of 0.001, 600 global epochs, a local batch size of 32, and 5 local epochs per client.

[1]https://pytorch.org/get-started/previous-versions/

[2]https://www.python.org/downloads/release/python-369/

[3]https://scikit-learn.org/stable/datasets/real_world.html#california-housing-dataset

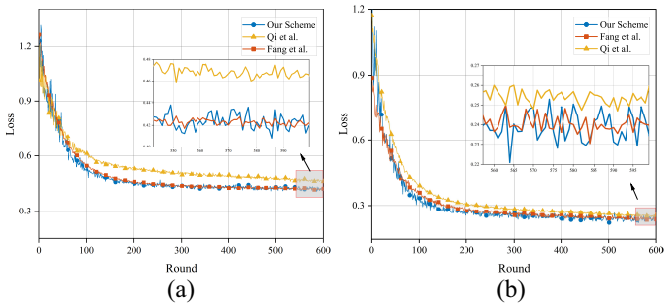


Fig. 2. Evaluation of model loss under different schemes as the number of clients varies. (a) Number of clients: 100. (b) Number of clients: 200.

3) Evaluation Metrics: We used absolute error (MAE), mean-square error (MSE), and root MSE (RMSE) to evaluate the model performance as follows:

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}\left(y_i - \hat{y}_p\right)^2 \tag{21}$$

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}\left|y_i - \hat{y}_p\right| \tag{22}$$

$$\text{RMSE} = \left[\frac{1}{n}\sum_{i=1}^{n}\left(\left|y_i - \hat{y}_p\right|\right)^2\right]^{\frac{1}{2}} \tag{23}$$

where y_i is the actual value, and $\hat{y}_p$ denotes the predicted value.

4) Baselines: We conducted comparative experiments with the following works.

1) *Qi et al. [41]:* This framework integrates DP with blockchain technology to provide strong privacy guarantees while enabling decentralized training.
2) *Fang et al. [43]:* This framework employs SA in combination with blockchain to enhance the security and privacy of FL. It further utilizes secret sharing to effectively manage client dropouts during training.

B. Performance Evaluation

Fig. 2 illustrates the loss comparison between our proposed method and baseline approaches under different client numbers. As the number of clients increases from 100 to 200, the model's loss decreases significantly, suggesting that the participation of more clients can accelerate convergence and improve model performance. It is worth noting that the loss of our scheme is similar to [43] and the significantly lower than [41], where the performance is affected by the introduction of DP-based noise. In contrast, the mask-based privacy-preserving mechanism adopted in this article achieves mask cancellation during the aggregation process, effectively protecting data privacy without sacrificing model performance.

Additionally, we further validated the effectiveness of the proposed scheme using three classical regression evaluation metrics: MSE, MAE, and RMSE, as detailed in Table III. Lower values of these metrics correspond to higher prediction accuracy. With 100 clients, the proposed scheme achieved

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:04:08 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPARISON OF MSE, MAE, AND RMSE FOR DIFFERENT SCHEMES

Schemes	Client Numbers = 100			Client Numbers = 200		
	MSE	MAE	RMSE	MSE	MAE	RMSE
Qi et al. [41]	0.567832	0.570454	0.753547	0.497557	0.521766	0.705377
Fang et al. [43]	0.504746	0.529400	0.710455	0.464685	0.502803	0.681678
Our scheme	0.491126	0.514460	0.700804	0.462701	0.502104	0.680221

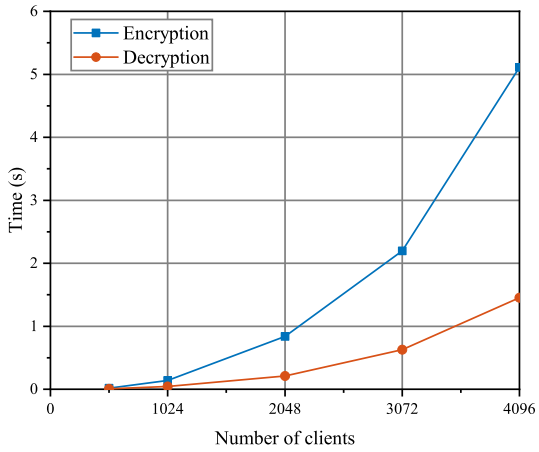


Fig. 3. Encryption and decryption computational cost with key size.

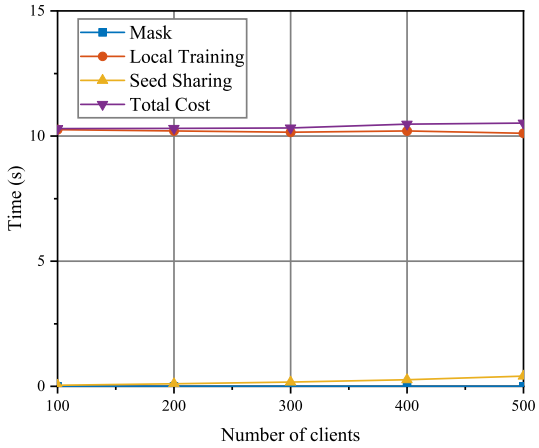


Fig. 4. Computation cost of local training in encrypted domains.

MSE, MAE, and RMSE values of 0.491126, 0.514460, and 0.700804, respectively, significantly outperforming [41]. When the number of clients increased to 200, the proposed scheme showed further improvement, with MSE dropping to 0.462701, MAE to 0.502104, and RMSE to 0.680221.

C. Computation Overhead Evaluation

In this section, we provide a detailed analysis of the computational overhead of our proposed scheme and compare it with [43]. Our analysis primarily include four critical aspects: 1) encryption and decryption operations; 2) local model training in encrypted domains; 3) aggregation and consensus operations; and 4) overall computational cost.

1) Encryption and Decryption Overhead: In our scheme, the TP is responsible for initializing the global model and encrypting its parameters using HE. Fig. 3 indicates that the running time for encrypting the global model and decrypting the aggregated model increases with the key size. For a 4096-bit key, encryption requires approximately 5 s, while decryption takes about 1.5 s. Unlike nonencrypted approaches such as [43], our scheme introduces computational overhead but provides essential privacy guarantees for FL. This security-efficiency tradeoff is crucial in resource-constrained IoT settings. Our analysis suggests that a 2048-bit key offers an optimal balance for most IoT applications, ensuring robust security with acceptable computational costs.

2) Local Model Training Overhead in Encrypted Domains: In our scheme, clients need to perform training in the ciphertext domain. Fig. 4 illustrates the computational cost of local training in encrypted domains, comprising three key components: 1) seed sharing; 2) local training; and 3) mask generation. Local training dominates the computational overhead due to the complex homomorphic operations required for secure computations on encrypted model. Seed sharing, as the second most time-consuming operation, sees its cost increase with the number of clients. In contrast, mask generation for the encrypted model incurs the least overhead, with its complexity primarily dependent on the size of model parameters rather than the number of participants. Considering these factors, the total computational cost on the client side exhibits a linear growth trend as the number of clients increases.

3) Aggregation and Consensus Cost: Fig. 5 illustrates the trends of the aggregation time and consensus time as the number of participating clients increases. Aggregation time refers to the time required for the leader to aggregate encrypted models from clients to generate a global model and construct a block, which increases with the number of clients. The reason is that as the number of clients increases, the leader needs to process more encrypted models, resulting in a corresponding increase in the computational burden of model aggregation. Consensus time reflects the time taken for the blockchain network to reach agreement on a new block, which remains relatively stable and is primarily related to the number of validator nodes rather than the number of clients. In our implementation, the number of validator nodes remains fixed, so the complexity of the consensus process does not significantly change with an increase in the number of clients.

4) Overall Computational Cost: The overall computational cost of our proposed scheme is determined by a combination

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:04:08 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
COMPUTATION TIME WITH DIFFERENT DROPOUT RATES

Clients	Dropout rate	Seed sharing	Local training	Aggregation	Consensus	Total
100	0%	46.2 ms	11814 ms	72.49 ms	5410 ms	17342.69 ms
	10%	46.59 ms	11300 ms	77.35 ms	5794 ms	17217.94 ms
	20%	47.42 ms	13249 ms	76.86 ms	5508 ms	18881.28 ms
	30%	45.9 ms	12716 ms	80.05 ms	5916 ms	18757.95 ms
200	0%	91.68 ms	12531 ms	102.68 ms	5300 ms	18025.36 ms
	10%	95.44 ms	12084 ms	118.05ms	5191 ms	17488.49 ms
	20%	94.26 ms	13118 ms	109.43ms	5203 ms	18524.69 ms
	30%	92.89 ms	13055 ms	110.20ms	5410 ms	18668.09 ms

TABLE V
COMMUNICATION OVERHEAD WITH DIFFERENT DROPOUT RATES

Clients	Dropout rate	Seed sharing	Local training	Aggregation	Consensus	Total
100	0%	57.42 KB	13.54 MB	215 KB	5M	18.81 M
	10%	57.42 KB	12.19 MB	193.5 KB	4.5 M	16.94 M
	20%	57.42 KB	10.85 MB	154.8 KB	3.6 M	14.66 M
	30%	57.42 KB	9.51 MB	108.36 KB	3.2 M	12.87 M
200	0%	148.12 KB	27.07 MB	440.32 KB	9.4 M	37.04 MB
	10%	148.12 KB	24.39 MB	396.29 KB	8.46 M	33.38 MB
	20%	148.12 KB	21.70 MB	317.03 KB	7.52 M	29.67 MB
	30%	148.12 KB	19.01 MB	221.92 KB	6.58 M	25.95 MB

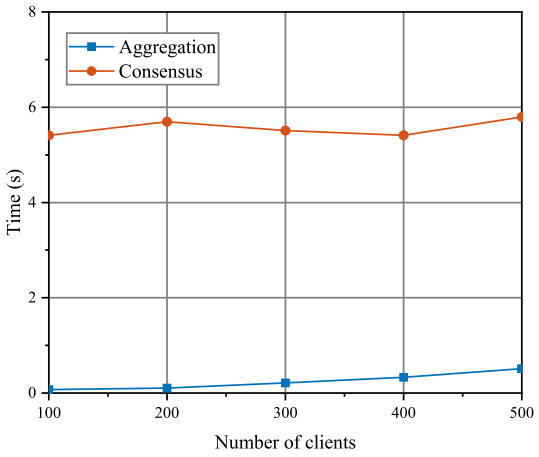


Fig. 5. Computation cost of aggregation and consensus.

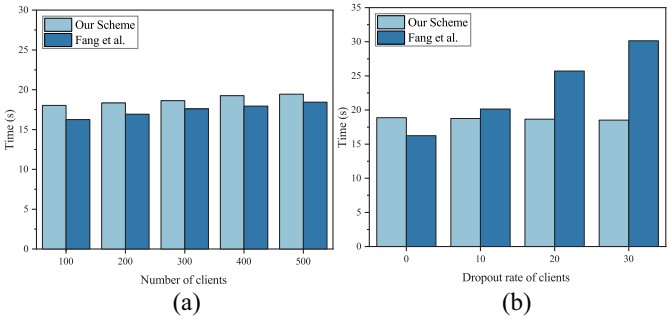


Fig. 6. Comparison of computation time with different dropout rates and client numbers. (a) Computation time versus number of clients. (b) Computation time versus dropout rate of clients.

of client-side operations (local training, seed sharing, and mask generation) and blockchain-side operations (aggregation and consensus), as illustrated in Table IV. Fig. 6 provides a comparative analysis of computation times between our method and [43] at different dropout rates and client numbers. We can make two key observations from this analysis: First, our scheme demonstrates remarkable resilience to increasing dropout rates, maintaining consistent computational overhead due to the mask generation process being independent of dropout users. This contrasts with [43], which may exhibit sensitivity to dropout rate variations. Second, while our scheme yields slightly higher computation time compared to [43], this difference is attributed to the additional overhead of encrypted domain training. In [43], the client can train the plaintext global model locally, thus avoiding the complexity of encryption, whereas our approach must train the model locally in the encrypted domain, introducing homomorphic computational overhead. Despite the slight increase in computation time, our approach provides stronger privacy protection by encrypting the model end to end, which is a crucial feature for privacy-sensitive applications, and thus a slight performance tradeoff is justified.

D. Communication Overhead Analysis

Table V presents the detailed communication overhead of the proposed mechanism under varying numbers of clients and dropout rates. The results indicate that communication overhead is primarily influenced by two factors: 1) the number of clients and 2) the dropout rate. As the number of clients increases,

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:04:08 UTC from IEEE Xplore. Restrictions apply.

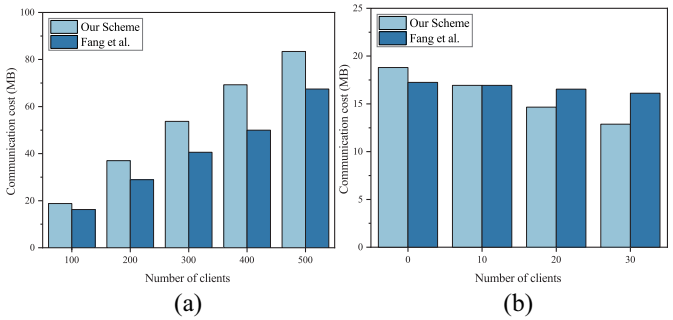


Fig. 7. Comparison of communication cost with different dropout rates and client numbers. (a) Communication cost versus number of clients. (b) Communication cost versus dropout rate of clients.

the overall communication overhead shows an upward trend. Conversely, as the dropout rate increases, the communication overhead exhibits a downward trend. For example, with 100 clients, when the dropout rate increases from 0% to 30%, the total communication overhead decreases from 18.81 to 12.87 MB, a reduction of approximately 31.6%. This phenomenon can be attributed to the fact that as the dropout rate increases, the number of clients actively participating in the computation decreases, directly reducing the number of local model updates that need to be transmitted and aggregated, thereby lowering the overall communication overhead.

Fig. 7 compares the communication overhead of our proposed scheme with that of [43] under different numbers of clients and dropout rates. Overall, our scheme exhibits slightly higher communication overhead compared to [43], primarily due to the ciphertext expansion resulting from the introduction of HE. Nevertheless, considering the importance of privacy protection, we believe this increase in communication overhead is within an acceptable range. In contrast, [43] employs the CRT to compress local gradients, thus achieving an advantage in communication volume.

However, it is noteworthy that as the dropout rate increases, our scheme demonstrates superior stability. Our communication volume does not increase significantly, whereas the communication volume in [43] shows an upward trend. This difference mainly stems from the two schemes employ different methods for handling client dropouts. Our scheme utilizes a single-mask mechanism, which ensures the correctness and efficiency of model aggregation even in the presence of client dropouts. In contrast, the double-mask mechanism adopted by [43] requires additional communication with online clients to coordinate the model aggregation process when clients drop out, especially during the mask removal phase. This extra coordination process leads to an increase in communication volume as the dropout rate rises. Consequently, our scheme exhibits advantages in handling client dynamics, particularly in scenarios with high dropout rates.

VIII. Conclusion

In this article, we propose a PBFL that enhances privacy and security by integrating HE and single masking techniques in IIoT environments. PBFL enables clients to train their local models securely within the ciphertext domain without requiring decryption of the global model. The single masking technique allows clients to apply unique random masks to their encrypted local model updates, enabling SA while preserving local model privacy. Blockchain technology is employed to facilitate decentralized aggregation and encrypted model storage, effectively addressing the risks of centralized server vulnerabilities. Experimental results demonstrate that the effectiveness of the proposed scheme in terms of computational and communication. However, the encryption of local models may increase the difficulty of detecting malicious models. Future work will focus on developing efficient anomaly detection methods suitable for encrypted computing environments. Additionally, we plan to apply PBFL to real-world scenarios, such as healthcare and intelligent transportation, to further validate its performance and applicability.

References

[1] S. Suhail, R. Hussain, R. Jurdak, and C. S. Hong, "Trustworthy digital twins in the Industrial Internet of Things with blockchain," *IEEE Internet Comput.*, vol. 26, no. 3, pp. 58–67, May/Jun. 2022.

[2] T. V. Khoa et al., "Deep transfer learning: A novel collaborative learning model for cyberattack detection systems in IoT networks," *IEEE Internet Things J.*, vol. 10, no. 10, pp. 8578–8589, May 2023.

[3] G. Xie et al., "GAI-IoV: Bridging generative AI and vehicular networks for ubiquitous edge intelligence," *IEEE Trans. Wireless Commun.*, vol. 23, no. 10, pp. 12799–12814, Oct. 2024.

[4] B. Yang et al., "A joint energy and latency framework for transfer learning over 5G industrial edge networks," *IEEE Trans. Ind. Informat.*, vol. 18, no. 1, pp. 531–541, Jan. 2022.

[5] T. V. Khoa et al., "Real-time cyberattack detection with collaborative learning for blockchain networks," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, 2024, pp. 1–6.

[6] A. Arunan, Y. Qin, X. Li, and C. Yuen, "A federated learning-based industrial health prognostics for heterogeneous edge devices using matched feature extraction," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 3, pp. 3065–3079, Jul. 2024.

[7] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, 2017, pp. 1273–1282.

[8] L. You, S. Liu, Y. Chang, and C. Yuen, "A triple-step asynchronous federated learning mechanism for client activation, interaction optimization, and aggregation enhancement," *IEEE Internet Things J.*, vol. 9, no. 23, pp. 24199–24211, Dec. 2022.

[9] Y. Miao, Z. Liu, H. Li, K.-K. R. Choo, and R. H. Deng, "Privacy-preserving Byzantine-robust federated learning via blockchain systems," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 2848–2861, 2022.

[10] H. Yang, J. Zhao, Z. Xiong, K.-Y. Lam, S. Sun, and L. Xiao, "Privacy-preserving federated learning for UAV-enabled networks: Learning-based joint scheduling and resource management," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 10, pp. 3144–3159, Oct. 2021.

[11] F. Naeem, M. Ali, G. Kaddoum, C. Huang, and C. Yuen, "Security and privacy for reconfigurable intelligent surface in 6G: A review of prospective applications and challenges," *IEEE Open J. Commun. Soc.*, vol. 4, pp. 1196–1217, 2023.

[12] S. Chen, X. Tan, W. Y. B. Lim, Z. Xiong, and H. Yu, "Privacy budget-aware incentive mechanism for federated learning in intelligent transportation systems," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2023, pp. 3060–3065.

[13] M. Fan, K. Ji, Z. Zhang, H. Yu, and G. Sun, "Lightweight privacy and security computing for blockchained federated learning in IoT," *IEEE Internet Things J.*, vol. 10, no. 18, pp. 16048–16060, Sep. 2023.

[14] X. Li, M. Wen, S. He, R. Lu, and L. Wang, "A privacy-preserving federated learning scheme against poisoning attacks in smart grid," *IEEE Internet Things J.*, vol. 11, no. 9, pp. 16805–16816, May 2024.

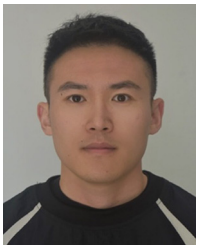
[15] A. Fu, X. Zhang, N. Xiong, Y. Gao, H. Wang, and J. Zhang, "VFL: A verifiable federated learning with privacy-preserving for big data in industrial IoT," *IEEE Trans. Ind. Informat.*, vol. 18, no. 5, pp. 3316–3326, May 2022.

[16] R. Wang, X. Yuan, Z. Yang, Y. Wan, M. Luo, and D. Wu, "RFLPV: A robust federated learning scheme with privacy preservation and verifiable aggregation in IoMT," *Inf. Fusion*, vol. 102, Feb. 2024, Art. no. 102029.

[17] G. Xu, H. Li, S. Liu, K. Yang, and X. Lin, "VerifyNet: Secure and verifiable federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 15, pp. 911–926, 2020.

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:04:08 UTC from IEEE Xplore. Restrictions apply.

[18] S. Lu, R. Li, W. Liu, C. Guan, and X. Yang, "Top-k sparsification with secure aggregation for privacy-preserving federated learning," *Comput. Secur.*, vol. 124, Jan. 2023, Art. no. 102993.
[19] M. B. Mollah et al., "Blockchain for the Internet of Vehicles towards intelligent transportation systems: A survey," *IEEE Internet Things J.*, vol. 8, no. 6, pp. 4157–4185, Mar. 2021.
[20] Z. Liu, J. Guo, K.-Y. Lam, and J. Zhao, "Efficient dropout-resilient aggregation for privacy-preserving machine learning," *IEEE Trans. Inf. Forensics Security*, vol. 18, pp. 1839–1854, 2022.
[21] J. Kang, Z. Xiong, D. Niyato, Y. Zou, Y. Zhang, and M. Guizani, "Reliable federated learning for mobile networks," *IEEE Wireless Commun.*, vol. 27, no. 2, pp. 72–80, Apr. 2020.
[22] Y. Liu, Z. Ai, S. Sun, S. Zhang, Z. Liu, and H. Yu, "Fedcoin: A peer-to-peer payment system for federated learning," in *Federated Learning: Privacy and Incentive*. Cham, Switzerland: Springer, 2020, pp. 125–138.
[23] Y. Ren, M. Hu, Z. Yang, G. Feng, and X. Zhang, "BPFL: Blockchain-based privacy-preserving federated learning against poisoning attack," *Inf. Sci.*, vol. 665, Apr. 2024, Art. no. 120377.
[24] Y. Wang, Z. Su, N. Zhang, and A. Benslimane, "Learning in the air: Secure federated learning for UAV-assisted crowdsensing," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1055–1069, Apr.–Jun. 2021.
[25] Z. Xiong, Z. Cai, D. Takabi, and W. Li, "Privacy threat and defense for federated learning with non-i.i.d. Data in AIoT," *IEEE Trans. Ind. Informat.*, vol. 18, no. 2, pp. 1310–1321, Feb. 2022.
[26] K. Wei et al., "User-level privacy-preserving federated learning: Analysis and performance optimization," *IEEE Trans. Mobile Comput.*, vol. 21, no. 9, pp. 3388–3401, Sep. 2022.
[27] T. V. Khoa, M. A. Alsheikh, Y. Alem, and D. T. Hoang, "Balancing security and accuracy: A novel federated learning approach for Cyberattack detection in blockchain networks," 2024, *arXiv:2409.04972*.
[28] X. Shen, Y. Liu, F. Li, and C. Li, "Privacy-preserving federated learning against label-flipping attacks on non-IID data," *IEEE Internet Things J.*, vol. 11, no. 1, pp. 1241–1255, Jan. 2024.
[29] C. Zhang, S. Li, J. Xia, W. Wang, F. Yan, and Y. Liu, "BatchCrypt: Efficient homomorphic encryption for cross-silo federated learning," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2020, pp. 493–506.
[30] Z. Ma, J. Ma, Y. Miao, Y. Li, and R. H. Deng, "ShieldFL: Mitigating model poisoning attacks in privacy-preserving federated learning," *IEEE Trans. Inf. Forensics Security*, vol. 17, pp. 1639–1654, 2022.
[31] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1175–1191.
[32] K. Mandal and G. Gong, "PrivFL: Practical privacy-preserving federated regressions on high-dimensional data over mobile networks," in *Proc. ACM SIGSAC Conf. Cloud Comput. Secur. Workshop*, 2019, pp. 57–68.
[33] P. Zhao, Z. Cao, J. Jiang, and F. Gao, "Practical private aggregation in federated learning against inference attack," *IEEE Internet Things J.*, vol. 10, no. 1, pp. 318–329, Jan. 2023.
[34] S. Yang, Y. Chen, Z. Yang, B. Li, and H. Liu, "Fast secure aggregation with high dropout resilience for federated learning," *IEEE Trans. Green Commun. Netw.*, vol. 7, no. 3, pp. 1501–1514, Sep. 2023.
[35] N. Ul Hassan, C. Yuen, and D. Niyato, "Blockchain technologies for smart energy systems: Fundamentals, challenges, and solutions," *IEEE Ind. Electron. Mag.*, vol. 13, no. 4, pp. 106–118, Dec. 2019.
[36] A. H. Khan et al., "Blockchain and 6G: The future of secure and ubiquitous communication," *IEEE Wireless Commun.*, vol. 29, no. 1, pp. 194–201, Feb. 2022.
[37] T. V. Khoa et al., "Collaborative learning for cyberattack detection in blockchain networks," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 54, no. 7, pp. 3920–3933, Jul. 2024.
[38] Y. Lu, X. Huang, K. Zhang, S. Maharjan, and Y. Zhang, "Blockchain empowered asynchronous federated learning for secure data sharing in Internet of Vehicles," *IEEE Trans. Veh. Technol.*, vol. 69, no. 4, pp. 4298–4311, Apr. 2020.
[39] C. Feng, B. Liu, K. Yu, S. K. Goudos, and S. Wan, "Blockchain-empowered decentralized horizontal federated learning for 5G-enabled UAVs," *IEEE Trans. Ind. Informat.*, vol. 18, no. 5, pp. 3582–3592, May 2022.
[40] B. Han et al., "Dynamic incentive design for federated learning based on consortium blockchain using a Stackelberg game," *IEEE Access*, vol. 12, pp. 160267–160283, 2024.
[41] Y. Qi, M. S. Hossain, J. Nie, and X. Li, "Privacy-preserving blockchain-based federated learning for traffic flow prediction," *Future Gener. Comput. Syst.*, vol. 117, pp. 328–337, 2021.
[42] H. Ma, S. Huang, J. Guo, K.-Y. Lam, and T. Yang, "Blockchain-based privacy-preserving federated learning for mobile crowdsourcing," *IEEE Internet Things J.*, vol. 11, no. 8, pp. 13884–13899, Apr. 2024.
[43] C. Fang, Y. Guo, J. Ma, H. Xie, and Y. Wang, "A privacy-preserving and verifiable federated learning method based on blockchain," *Comput. Commun.*, vol. 186, pp. 1–11, Mar. 2022.
[44] P. Paillier, "Public-key cryptosystems based on composite degree residuosity classes," in *Proc. Int. Conf. Theory Appl. Cryptogr. Techn.*, 1999, pp. 223–238.
[45] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, no. 11, pp. 612–613, 1979.
[46] W. Diffie and M. E. Hellman, "New directions in cryptography," in *Democratizing Cryptography: The Work of Whitfield Diffie and Martin Hellman*, New York, NY, USA: Association for Computing Machinery, 2022, pp. 365–390.
[47] J. Håstad, R. Impagliazzo, L. A. Levin, and M. Luby, "A pseudorandom generator from any one-way function," *SIAM J. Comput.*, vol. 28, no. 4, pp. 1364–1396, 1999.
[48] A. Banerjee, G. Fuchsbauer, C. Peikert, K. Pietrzak, and S. Stevens, "Key-homomorphic constrained pseudorandom functions," in *Proc. Theory Cryptogr. Conf.*, 2015, pp. 31–60.
[49] D. Boneh, "The decision Diffie–Hellman problem," in *Proc. Int. Algorithmic Number Theory Symp.*, 1998, pp. 48–63.
[50] Z. Liu, J. Guo, W. Yang, J. Fan, K.-Y. Lam, and J. Zhao, "Privacy-preserving aggregation in federated learning: A survey," *IEEE Trans. Big Data*, early access, Jul. 15, 2022, doi: 10.1109/TBDATA.2022.3190835.



Baofu Han received the M.S. degree from the School of Information Science and Engineering, Shenyang University of Technology, Shenyang, China, in 2021. He is currently pursuing the Ph.D. degree with the School of Cyber Science and Engineering, Southeast University, Nanjing, China.

His research interests include blockchain, federated learning, game theory, data privacy preservation, and their applications in the Internet of Things, intelligent transportation systems, and smart cities.



Bing Li was born in Nanjing, China, in 1968. He received the B.S. degree in electronics science and technology and the Ph.D. degree from Southeast University, Nanjing, China, in 1991 and 2004, respectively.

He is currently a Professor and a Tutor of doctoral students with the School of Microelectronics and the School of Cyber Science and Engineering, Southeast University, and the Director of the Joint Research Center for Advanced Cloud System. His main research is the efficient and secure integrated circuits and system, including data compression, data encryption, physical unclonable functions, blockchain, Internet of Things, and the area of information security.



Raja Jurdak (Senior Member, IEEE) received the B.E. degree in computer and communications engineering from the American University of Beirut, Beirut, Lebanon, in 2000, the M.S. degree in computer networks and distributed computing from the Department of Electrical and Computer Engineering, University of California at Irvine, Irvine, CA, USA, in 2001, and the Ph.D. degree in information and computer science from the University of California at Irvine, Irvine, in 2005.

He is a Professor of Distributed Systems and the Chair of Applied Data Sciences with Queensland University of Technology, Brisbane, QLD, Australia, where he is also the Director of the Trusted Networks Laboratory. He is also an Adjunct Professor with the University of New South Wales, Sydney, NSW, Australia, and a Visiting Researcher with Data61, CSIRO, Brisbane. Previously, he established and led the Distributed Sensing Systems Group, Data61, CSIRO. He also spent time as a Visiting Academic with Massachusetts Institute of Technology, Cambridge, MA, USA, and Oxford University, Oxford, U.K., in 2011 and 2017, respectively. He has published over 250 peer-reviewed publications, including three authored books on IoT, blockchain, and cyber–physical systems. His publications have attracted over 14 700 citations, with an H-index of 54. His research interests include trust, mobility, and energy efficiency in networks.

Prof. Jurdak serves on the organizing and technical program committees of top international conferences, including Percom, ICBC, IPSN, WoWMoM, and ICDCS. He serves on the editorial boards of *Ad Hoc Networks* and *Scientific Reports* (Nature). He is an IEEE Computer Society Distinguished Visitor.

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:04:08 UTC from IEEE Xplore. Restrictions apply.



Peiyun Zhang (Senior Member, IEEE) received the Ph.D. degree in computer science from the School of Computer Science and Technology, Nanjing University of Science and Technology, Nanjing, China, in 2008.

She is currently a Professor with the Engineering Research Center of Digital Forensics of Ministry of Education and the School of Computer Science, Nanjing University of Information Science and Technology, Nanjing. Her interests include blockchains and cloud/trust computing. She has published over 70 papers in these fields.



Hao Zhang was born in Ningbo, Zhejiang, China, in 2000. He received the B.S. degree from Zhejiang Gongshang University, Hangzhou, China, in 2022. He is currently pursuing the M.S. degree with the School of Microelectronics, Southeast University, Nanjing, China.

His research interests mainly include hardware security, especially for the physical unclonable functions.



Pan Feng was born in Shanxi, China, in 1995. She received the M.S. degree in computer technology from Henan University of Technology, Zhengzhou, China, in 2021. She is currently pursuing the Ph.D. degree with the School of Cyber Science and Engineering, Southeast University, Nanjing, Jiangsu, China.

Her research interests include information security, with a focus on blockchain and IoT technologies.



Chau Yuen (Fellow, IEEE) received the B.Eng. and Ph.D. degrees from Nanyang Technological University, Singapore, in 2000 and 2004, respectively.

He was a Postdoctoral Fellow with Lucent Technologies Bell Laboratories, Murray Hill, NJ, USA, in 2005. From 2006 to 2010, he was with the Institute for Infocomm Research, Singapore. From 2010 to 2023, he was with the Engineering Product Development Pillar, Singapore University of Technology and Design, Singapore. Since 2023, he has been with the School of Electrical and Electronic Engineering, Nanyang Technological University, where he is currently the Provost's Chair of Wireless Communications and the Assistant Dean of Graduate College. He has four U.S. patents and published over 400 research papers in international journals or conferences.

Dr. Yuen received the IEEE Communications Society Leonard G. Abraham Prize in 2024, the IEEE Communications Society Best Tutorial Paper Award in 2024, the IEEE Communications Society Fred W. Ellersick Prize in 2023, the IEEE Marconi Prize Paper Award in Wireless Communications in 2021, the IEEE APB Outstanding Paper Award in 2023, and the EURASIP Best Paper Award for Journal on Wireless Communications and Networking in 2021. He currently serves as the Editor-in-Chief for *Computer Science* (Springer Nature) and an Editor for IEEE TRANSACTIONS ON VEHICULAR TECHNOLOGY, IEEE SYSTEM JOURNAL, and IEEE TRANSACTIONS ON NETWORK SCIENCE AND ENGINEERING, where he was awarded the IEEE Transactions on Network Science and Engineering Excellent Editor Award and a Top Associate Editor for IEEE Transactions on Vehicular Technology from 2009 to 2015. He also served as a Guest Editor for several special issues, including IEEE JOURNAL ON SELECTED AREAS IN COMMUNICATIONS, *IEEE Wireless Communications Magazine*, *IEEE Communications Magazine*, *IEEE Vehicular Technology Magazine*, IEEE TRANSACTIONS ON COGNITIVE COMMUNICATIONS AND NETWORKING, and *Applied Energy* (Elsevier). He is a Distinguished Lecturer of the IEEE Vehicular Technology Society, a top 2% Scientist by Stanford University, and also a Highly Cited Researcher by Clarivate Web of Science.

Authorized licensed use limited to: Southeast University. Downloaded on October 18,2026 at 05:04:08 UTC from IEEE Xplore. Restrictions apply.